

HyMeKo Generation & Emission Framework

Multi-Target Codegen from a Single Hypergraph IR

thesis-IV working notes (HyMeKo)

2026-04-26

Abstract

HyMeKo describes robotic systems and neural-network architectures in a single hypergraph IR. From the same description it can emit URDF, SDF 1.7, MJCF, DOT, Mermaid, Gazebo world XML, and torch_dataflow PyTorch source. Each target is realised as a `(queries.hymeko, template.X)` pair under `transforms/<target>/`; the Rust template engine dispatches queries against the IR and renders the template. We document the architecture, the recently-added Tier 1 (plain MLP) and Tier 2 (residual / highway) emission paths for the PyTorch backend, and the publication venues this systems-level contribution targets separately from the entropy-regulariser paper.

1 What can HyMeKo emit?

Eight emission targets are currently shipped under `transforms/`:

Target	Purpose	Status
<code>urdf</code>	ROS robot description	shipped
<code>sdf</code>	Gazebo scene description	shipped
<code>mjcf</code>	MuJoCo XML	shipped
<code>gazebo</code>	Full Gazebo world XML	shipped
<code>dot</code>	Graphviz hypergraph viz	shipped
<code>mermaid</code>	Markdown-renderable graph	shipped
<code>ros2\launch</code>	ROS 2 launch.py file	shipped
<code>torch\dataflow</code>	PyTorch <code>nn.Module</code>	shipped + Tier 1 + Tier 2 (this report)

A single `.hymeko` source can be compiled to any subset:

```
hymeko compile data/robotics/anthropomorphic_arm.hymeko \  
  --format urdf -o /tmp/arm.urdf  
  
hymeko compile data/nn/mnist_small.hymeko \  
  --format torch_dataflow --name MnistSmall -o /tmp/mnist_small.py
```

2 Architecture: queries + template per target

Every target is a directory `transforms/<target>/` with two files:

[leftmargin=1.4em, style=nextline, itemsep=2pt]Declares named collections of decls matched by inheritance predicate. Example for `torch\dataflow`:

```
queries.hymeko: meta_layer      {}  
linear_layers: linear_layer    {}  
relu_layers:   relu_layer      {}  
residual_blocks: residual_block {}  
@flows:        dataflow        {}  
outputs:       t_output         {}
```

Each line `name: type {}` becomes a collection that the template can iterate.

template.<ext> Render-time template using Handlebars-like syntax: `\{\{config:robot\_name\}\}` for config args, `\{\{#each linear\_layers\}\}` to iterate matches, `\{\{name\}\}` / `\{\{field:d\_in\}\}` for per-decl values, `\{\{bind:+:0\}\}` for hyperedge arc bindings.

This separation has three consequences:

- Adding a new emission target = adding a new directory. No changes to Rust code.
- Per-format expressivity is constrained to what the template engine + queries can express jointly. Multi-input dataflow and conditional emission per inheritance branch are currently supported; arithmetic on field values is not.
- The IR remains the single source of truth: structural consistency across emitted targets is automatic (a robot described in HyMeKo will have the same kinematic chain in its URDF and its MJCF and its DOT).

3 Recent extension: Tier 1 + Tier 2 PyTorch dataflow

Until 2026-04-26 the `torch\_dataflow` target only emitted `HypergraphConv` layers — HGNN-style networks. We extended it to cover plain MLPs and residual / highway architectures so the networks tested by the entropy-regulariser benchmark can also be generated end-to-end from a HyMeKo description.

3.1 Tier 1 — plain MLP primitives

New `meta\_layer` subtypes added to `data/nn/meta\_nn.hymeko`:

- `linear\_layer` — emits `torch.nn.Linear(d\_in, d\_out)`
- `relu\_layer` — emits `torch.nn.ReLU()`
- `sigmoid\_layer` — emits `torch.nn.Sigmoid()`
- `tanh\_layer` — emits `torch.nn.Tanh()`

Example (`data/nn/mnist\_small.hymeko`, 32 lines) emits the same $784 \rightarrow 16 \rightarrow 8 \rightarrow 10$ MLP that `run\_benchmark.py`'s `MNISTNetSmall` hand-codes. Parameter count: 12 786 in both implementations (verified).

3.2 Tier 2 — composite blocks

For residual and gated networks, a fully-structural HyMeKo description would need multi-input dataflow hyperedges (the additive skip needs two incoming edges). That feature is deferred. Instead, Tier 2 adds **named composite primitives** whose internal structure is realised by helper classes shipped in `python/ehk\_torch\_stub`:

- `residual\_block` — emits `ResidualBlock(hidden=...)`; helper class implements $y = x + \text{Linear}(\text{ReLU}(\text{Linear}(x)))$.
- `highway\_block` — emits `HighwayBlock(hidden=...)`; helper implements $y = T(x) \odot F(x) + (1 - T(x)) \odot x$.

Example (`data/nn/mnist\_resmlp\_3.hymeko`, 35 lines) emits a 3-block ResMLP equivalent to `ResMLP(hidden=16, n\_blocks=3)`. Parameter count: 14 362 in both implementations (verified).

3.3 Why named-block instead of full-structural

Three reasons:

1. **Engineering scope.** Multi-input dataflow hyperedges are a real change to the template’s iteration semantics (each `@flow` currently has a fixed (input, layer, output) shape). Adding multi-input is feasible but non-trivial.
2. **Architectural clarity.** A named block is more readable than the equivalent hypergraph topology with explicit skip vertices. Domain experts describing networks at the `.hymeko` level want to write `block\_1: lyr.residual\_block { hidden 16; }`, not unwind the additive skip into hypergraph primitives.
3. **Round-trippability.** Named blocks survive the round-trip from `.hymeko` $\rightarrow$ PyTorch $\rightarrow$ inspection. A flattened topology would lose the “this is a residual” annotation needed for higher-level analysis.

4 Verification

Description (lines)	Emitted module	Hand-coded equivalent	Param count match
<code>mnist_small.hymeko</code> (32)	<code>MnistSmallFromHymeko</code>	<code>MNISTNetSmall</code>	12 786 / 12 786
<code>mnist_resmlp_3.hymeko</code> (35)	<code>ResMLP3FromHymeko</code>	<code>ResMLP(16, 3)</code>	14 362 / 14 362

The emitted networks instantiate cleanly and forward-pass end-to-end on a 4×784 input batch, returning the expected 4×10 output.

5 What this is not yet

- **No CapsMLP emission.** Dynamic routing iterations require either inline Python loop emission or a fully opaque `CapsuleRouting` helper. Both are doable; deferred.
- **No batch-norm / dropout emission.** These need to be decided at the `linear\_layer` level (annotation field?) or as separate single-purpose layer types (`batchnorm\_layer`, `dropout\_layer`). Trivial to add when the use case requires.
- **No multi-input dataflow hyperedges.** Required for fully-structural ResNet / Highway / U-Net descriptions. Targeted for v0.3.
- **No regulariser-aware emission.** The emitted PyTorch modules don’t expose the `spectral\_weights()` method the entropy regulariser needs. Could be auto-emitted from the `linear\_layer` list. Trivial; deferred.

6 Publication venues for the emission framework

Distinct from the entropy-regulariser paper (which is a methods/ optimisation contribution at TMLR/NeurIPS/AIS scope). The generation/emission framework is a *language and systems* contribution. Realistic venues:

[leftmargin=1.4em, style=nextline, itemsep=2pt]Already submitting the hypergraph IR for robotic systems. Tier 1 / Tier 2 PyTorch emission can be added as a methods extension or a follow-up application paper. Short-form journal explicitly for computer-science software contributions. The multi-target codegen architecture fits the format exactly. If we publish a version of HyMeKo as a standalone open-source release. Short paper format (≈ 1000 words), the emphasis is on the software’s utility and reproducibility. Domain-specific languages and model-driven engineering. The HyMeKo grammar + emission targets fit this audience well. For the robotic- systems angle (URDF / SDF / MJCF emission). The Galambos student work on VLM-driven robot descriptions could feed an ICRA workshop submission. The PyTorch emission story (especially with regulariser-aware emission added) is a reasonable workshop fit.

Strongest recommendation: Software Impacts plus a JOSS paper in parallel. Both are short, both reward reproducibility (already met by the workspace’s reproducible-test infrastructure), and both

provide citable artefacts that downstream ML/robotics work can refer to. The MDPI Technologies and ICRA-workshop venues are application papers built on top of this systems contribution.

7 Architecture summary

The full emission stack (single direction):

```
.hymeko source
|
| parser (LALRPOP, lexer with SIMD)
v
AST (parser::ast::AstStr<'a>)
|
| hymeko_core::module_store::ModuleStore::compile
v
Compiled IR (CompiledProgram: nodes, edges, arcs, decl tree, hash)
|
| hymeko_query::transforms::TransformRegistry
| + transforms/<target>/queries.hymeko
v
Per-format query results (Vec<QueryMatch>)
|
| template engine
| + transforms/<target>/template.<ext>
v
Emitted artefact (URDF, MJCF, .py, .dot, ...)
```

The hot extension surface (where Tier 1 / Tier 2 changes landed) is on the right edge: the `queries.hymeko` and `template.py` files for the `torch\_dataflow` format. Zero changes to Rust code; the extension is purely declarative.

8 Reproducibility

To regenerate both example emissions from this workspace:

```
target/release/hymeko compile data/nn/mnist_small.hymeko \
  --format torch_dataflow --name MnistSmallFromHymeko \
  -o /tmp/mnist_small_emitted.py

target/release/hymeko compile data/nn/mnist_resmlp_3.hymeko \
  --format torch_dataflow --name ResMLP3FromHymeko \
  -o /tmp/mnist_resmlp_emitted.py

PYTHONPATH=python/ehk_torch_stub/src python3 -c "
import torch, importlib.util, sys
def load(p, n):
    s = importlib.util.spec_from_file_location(n, p)
    m = importlib.util.module_from_spec(s); s.loader.exec_module(m); return m
m1 = load('/tmp/mnist_small_emitted.py', 'm1').MnistSmallFromHymeko()
m2 = load('/tmp/mnist_resmlp_emitted.py', 'm2').ResMLP3FromHymeko()
print('p1 =', sum(p.numel() for p in m1.parameters()),
      ', p2 =', sum(p.numel() for p in m2.parameters()))
"
# Expected:  p1 = 12786 , p2 = 14362
```